1. Array implementation of stacks

```
#include <iostream>
using namespace std;

int arr[100];
int top = -1;
int SIZE;

void push(int val)
{
   if(top == SIZE - 1)
      cout << "Stack is Full" << endl;
   else
   {
      ++top;
      arr[top] = val;
      cout << val << " inserted successfully." << endl;
   }
}

void pop()
{
   if(top == -1)
      cout << "Stack is Empty" << endl;
   else
   {
      cout << "Popped element = " << arr[top] << endl;
      top--;
   }
}

void peek()
{
   if(top == -1)
      cout << "Stack is Empty" << endl;
   else
      cout << "Top Element = " << arr[top] << endl;
}

void display()
{
   if(top == -1)
      cout << "Stack is Empty" << endl;
   else
   {
      cout << "Stack Elements: ";

      for(int i = top; i >= 0; i--)
         cout << arr[i] << " ";
```

```
        cout << endl;
    }
}

int main()
{
    int choice, value;

    cout << "Enter Stack Size: ";
    cin >> SIZE;

    do
    {
        cout << "\n===== Stack Menu =====\n";
        cout << "1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n";
        cout << "Enter Choice: ";
        cin >> choice;

        switch(choice)
        {
            case 1:
                cout << "Enter Element: ";
                cin >> value;
                push(value);
                break;

            case 2: pop(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: cout << "Program Terminated.\n"; break;
            default: cout << "Invalid Choice!\n";
        }

    } while(choice != 5);

    return 0;
}
```

2. Array implementation of Queues

```
#include <iostream>
using namespace std;

int arr[100];
int front = 0;
int rear = 0;
int SIZE;

void enqueue(int val)
{
   if(rear == SIZE)
      cout << "Queue is Full" << endl;
   else
   {
      arr[rear] = val;
      rear++;
      cout << val << " inserted successfully." << endl;
   }
}

void dequeue()
{
   if(front == rear)
      cout << "Queue is Empty" << endl;
   else
   {
      cout << "Dequeued element = " << arr[front] << endl;
      front++;
   }
}

void peek()
{
   if(front == rear)
      cout << "Queue is Empty" << endl;
   else
      cout << "Front Element = " << arr[front] << endl;
}

void display()
{
   if(front == rear)
      cout << "Queue is Empty" << endl;
   else
   {
      cout << "Queue Elements: ";

      for(int i = front; i < rear; i++)
         cout << arr[i] << " ";
```

```
        cout << endl;
    }
}

int main()
{
    int choice, value;

    cout << "Enter Queue Size: ";
    cin >> SIZE;

    do
    {
        cout << "\n===== Queue Menu =====\n";
        cout << "1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. Exit\n";
        cout << "Enter Choice: ";
        cin >> choice;

        switch(choice)
        {
            case 1:
                cout << "Enter Element: ";
                cin >> value;
                enqueue(value);
                break;

            case 2: dequeue(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: cout << "Program Terminated.\n"; break;
            default: cout << "Invalid Choice!\n";
        }

    } while(choice != 5);

    return 0;
}
```

3. Linked list implementation of stacks

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *next;
};

Node *head = NULL;

void push(int val)
{
   Node *newNode = new Node();

   newNode->data = val;
   newNode->next = head;
   head = newNode;

   cout << val << " inserted successfully." << endl;
}

void pop()
{
   if(head == NULL)
      cout << "Stack is Empty" << endl;
   else
   {
      cout << "Popped element = " << head->data << endl;

      Node *temp = head;
      head = head->next;
      delete temp;
   }
}

void peek()
{
   if(head == NULL)
      cout << "Stack is Empty" << endl;
   else
      cout << "Top Element = " << head->data << endl;
}

void display()
{
   if(head == NULL)
      cout << "Stack is Empty" << endl;
   else
   {
      cout << "Stack Elements: ";

      Node *temp = head;
```

```
        while(temp != NULL)
        {
            cout << temp->data << " ";
            temp = temp->next;
        }

        cout << endl;
    }
}

int main()
{
    int choice, value;

    do
    {
        cout << "\n===== Stack Menu =====\n";
        cout << "1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n";
        cout << "Enter Choice: ";
        cin >> choice;

        switch(choice)
        {
            case 1:
                cout << "Enter Element: ";
                cin >> value;
                push(value);
                break;

            case 2: pop(); break;
            case 3: peek(); break;
            case 4: display(); break;
            case 5: cout << "Program Terminated.\n"; break;
            default: cout << "Invalid Choice!\n";
        }

    } while(choice != 5);

    return 0;
}
```

4. Linked list implementation of Queues

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *next;
};

Node *front = NULL;
Node *rear = NULL;

void enqueue(int val)
{
   Node *newNode = new Node();

   newNode->data = val;
   newNode->next = NULL;

   if(front == NULL && rear == NULL)
   {
      front = rear = newNode;
   }
   else
   {
      rear->next = newNode;
      rear = newNode;
   }

   cout << val << " inserted successfully." << endl;
}

void dequeue()
{
   if(front == NULL)
      cout << "Queue is Empty" << endl;
   else
   {
      cout << "Dequeued element = " << front->data << endl;

      Node *temp = front;
      front = front->next;

      if(front == NULL)
         rear = NULL;

      delete temp;
   }
}
```

```
void peek()
{
   if(front == NULL)
      cout << "Queue is Empty" << endl;
   else
      cout << "Front Element = " << front->data << endl;
}

void display()
{
   if(front == NULL)
      cout << "Queue is Empty" << endl;
   else
   {
      cout << "Queue Elements: ";

      Node *temp = front;

      while(temp != NULL)
      {
         cout << temp->data << " ";
         temp = temp->next;
      }

      cout << endl;
   }
}

int main()
{
   int choice, value;

   do
   {
      cout << "\n===== Queue Menu =====\n";
      cout << "1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. Exit\n";
      cout << "Enter Choice: ";
      cin >> choice;

      switch(choice)
      {
         case 1:
            cout << "Enter Element: ";
            cin >> value;
            enqueue(value);
            break;

         case 2: dequeue(); break;
         case 3: peek(); break;
```

```
            case 4: display(); break;
            case 5: cout << "Program Terminated.\n"; break;
            default: cout << "Invalid Choice!\n";
        }

    } while(choice != 5);

    return 0;
}
```

5. Infix to Postfix Expression Conversion

```
#include <iostream>
#include <cctype>
using namespace std;

char stack[100];
int top = -1;

void push(char ch)
{
   top++;
   stack[top] = ch;
}

char pop()
{
   char ch = stack[top];
   top--;
   return ch;
}

int priority(char ch)
{
   if(ch == '^')
      return 3;
   else if(ch == '*' || ch == '/')
      return 2;
   else if(ch == '+' || ch == '-')
      return 1;
   else
      return 0;
}

int main()
{
   char infix[100], postfix[100];
   int choice;

   do
   {
      cout << "\n===== Infix to Postfix =====\n";
      cout << "1. Convert Expression\n";
      cout << "2. Exit\n";
      cout << "Enter Choice: ";
      cin >> choice;

      if(choice == 1)
      {
         int i = 0, j = 0;
         top = -1;
```

```
cout << "Enter Infix Expression: ";
cin >> infix;

while(infix[i] != '\0')
{
   char ch = infix[i];

   if(isalnum(ch))
   {
      postfix[j] = ch;
      j++;
   }
   else if(ch == '(')
   {
      push(ch);
   }
   else if(ch == ')')
   {
      while(top != -1 && stack[top] != '(')
      {
         postfix[j] = pop();
         j++;
      }

      pop();
   }
   else
   {
      while(top != -1 &&
            stack[top] != '(' &&
            priority(stack[top]) >= priority(ch))
      {
         postfix[j] = pop();
         j++;
      }

      push(ch);
   }

   i++;
}

while(top != -1)
{
   postfix[j] = pop();
   j++;
}

postfix[j] = '\0';
```

```
            cout << "Postfix Expression: " << postfix << endl;
        }
        else if(choice == 2)
        {
            cout << "Program Terminated." << endl;
        }
        else
        {
            cout << "Invalid Choice!" << endl;
        }

    } while(choice != 2);

    return 0;
}
```

6. Binary Tree Traversals (Inorder, Preorder, Postorder)

```
#include <iostream>
using namespace std;

struct Node
{
   int data;
   Node *left;
   Node *right;
};

Node* createTree()
{
   int value;

   cout << "Enter value (-1 for no node): ";
   cin >> value;

   if(value == -1)
      return NULL;

   Node *newNode = new Node();
   newNode->data = value;

   cout << "Enter left child of " << value << endl;
   newNode->left = createTree();

   cout << "Enter right child of " << value << endl;
   newNode->right = createTree();

   return newNode;
}

void inorder(Node *root)
{
   if(root != NULL)
   {
      inorder(root->left);
      cout << root->data << " ";
      inorder(root->right);
   }
}

void preorder(Node *root)
{
   if(root != NULL)
   {
      cout << root->data << " ";
      preorder(root->left);
```

```
        preorder(root->right);
    }
}

void postorder(Node *root)
{
    if(root != NULL)
    {
        postorder(root->left);
        postorder(root->right);
        cout << root->data << " ";
    }
}

int main()
{
    Node *root;

    cout << "Create Binary Tree\n";
    root = createTree();

    cout << "\nInorder Traversal: ";
    inorder(root);

    cout << "\nPreorder Traversal: ";
    preorder(root);

    cout << "\nPostorder Traversal: ";
    postorder(root);

    return 0;
}
```

7. Linear Search and Binary Search Implementation

```
#include <iostream>
using namespace std;

void sortArray(int a[], int n)
{
   for(int i = 0; i < n - 1; i++)
      for(int j = 0; j < n - i - 1; j++)
         if(a[j] > a[j + 1])
         {
            int t = a[j];
            a[j] = a[j + 1];
            a[j + 1] = t;
         }
}

int main()
{
   int a[100], n, key, choice, pos, count;

   cout << "Enter number of elements: ";
   cin >> n;

   cout << "Enter elements: ";
   for(int i = 0; i < n; i++)
      cin >> a[i];

   do
   {
      cout << "\n\n===== Search Menu =====";
      cout << "\n1. Linear Search - Unsorted Array";
      cout << "\n2. Linear Search - Sorted Array";
      cout << "\n3. Binary Search - Sorted Array";
      cout << "\n4. Exit";
      cout << "\nEnter Choice: ";
      cin >> choice;

      if(choice == 1)
      {
         cout << "\nEnter element to search: ";
         cin >> key;

         count = 0;

         for(int i = 0; i < n; i++)
         {
            if(a[i] == key)
            {
               cout << "Element found at position " << i + 1 << endl;
```

```
            count++;
        }
    }

    if(count == 0)
        cout << "Element not found.";
    else
        cout << "Element occurs " << count << " time(s).";
}

else if(choice == 2)
{
    sortArray(a, n);

    cout << "\nSorted Array: ";
    for(int i = 0; i < n; i++)
        cout << a[i] << " ";

    cout << "\nEnter element to search: ";
    cin >> key;

    count = 0;

    for(int i = 0; i < n; i++)
    {
        if(a[i] == key)
        {
            cout << "Element found at position " << i + 1 << endl;
            count++;
        }
    }

    if(count == 0)
        cout << "Element not found.";
    else
        cout << "Element occurs " << count << " time(s).";
}

else if(choice == 3)
{
    sortArray(a, n);

    cout << "\nSorted Array: ";
    for(int i = 0; i < n; i++)
        cout << a[i] << " ";

    cout << "\nEnter element to search: ";
    cin >> key;

    int low = 0, high = n - 1;
```

```
            pos = -1;

            while(low <= high)
            {
               int mid = (low + high) / 2;

               if(a[mid] == key)
               {
                  pos = mid;
                  break;
               }
               else if(a[mid] < key)
                  low = mid + 1;
               else
                  high = mid - 1;
            }

            if(pos == -1)
               cout << "Element not found.";
            else
               cout << "Element found at position " << pos + 1;
         }

         else if(choice == 4)
            cout << "Program Terminated.";

         else
            cout << "Invalid Choice.";

      } while(choice != 4);

      return 0;
   }
```

8. Graph Traversal using Depth First Search (DFS) and Breadth First Search (BFS)

```
#include <iostream>
using namespace std;

int graph[100][100];
int visited[100];
int n;

void DFS(int vertex)
{
   cout << vertex << " ";
   visited[vertex] = 1;

   for(int i = 1; i <= n; i++)
   {
      if(graph[vertex][i] == 1 && visited[i] == 0)
         DFS(i);
   }
}

void BFS(int start)
{
   int queue[100];
   int front = 0, rear = 0;

   for(int i = 1; i <= n; i++)
      visited[i] = 0;

   queue[rear] = start;
   rear++;
   visited[start] = 1;

   while(front < rear)
   {
      int vertex = queue[front];
      front++;

      cout << vertex << " ";

      for(int i = 1; i <= n; i++)
      {
         if(graph[vertex][i] == 1 && visited[i] == 0)
         {
            queue[rear] = i;
            rear++;
            visited[i] = 1;
         }
```

```
        }
    }
}

int main()
{
    int choice, start;

    cout << "Enter number of vertices: ";
    cin >> n;

    cout << "Enter adjacency matrix:\n";

    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= n; j++)
            cin >> graph[i][j];
    }

    do
    {
        cout << "\n\n===== Graph Traversal Menu =====";
        cout << "\n1. Depth First Search (DFS)";
        cout << "\n2. Breadth First Search (BFS)";
        cout << "\n3. Exit";
        cout << "\nEnter Choice: ";
        cin >> choice;

        if(choice == 1)
        {
            for(int i = 1; i <= n; i++)
                visited[i] = 0;

            cout << "Enter starting vertex: ";
            cin >> start;

            cout << "DFS Traversal: ";
            DFS(start);
        }

        else if(choice == 2)
        {
            cout << "Enter starting vertex: ";
            cin >> start;

            cout << "BFS Traversal: ";
            BFS(start);
        }

        else if(choice == 3)
```

```
            cout << "Program Terminated.";

        else
            cout << "Invalid Choice.";

    } while(choice != 3);

    return 0;
}
```

9. Finding single source shortest path of a Graph

```
#include <iostream>
using namespace std;

int graph[100][100];
int dist[100];
int visited[100];
int n;

void dijkstra(int start)
{
   for(int i = 1; i <= n; i++)
   {
      dist[i] = 9999;
      visited[i] = 0;
   }

   dist[start] = 0;

   for(int count = 1; count <= n; count++)
   {
      int minimum = 9999;
      int vertex = -1;

      for(int i = 1; i <= n; i++)
      {
         if(visited[i] == 0 && dist[i] < minimum)
         {
            minimum = dist[i];
            vertex = i;
         }
      }

      if(vertex == -1)
         break;

      visited[vertex] = 1;

      for(int i = 1; i <= n; i++)
      {
         if(visited[i] == 0 &&
            graph[vertex][i] != 0 &&
            dist[vertex] + graph[vertex][i] < dist[i])
         {
            dist[i] = dist[vertex] + graph[vertex][i];
         }
      }
```

```
    }

    cout << "\nShortest Distances from Vertex " << start << ":\n";

    for(int i = 1; i <= n; i++)
    {
        if(dist[i] == 9999)
            cout << start << " -> " << i << " = No Path" << endl;
        else
            cout << start << " -> " << i << " = "
                 << dist[i] << endl;
    }
}

int main()
{
    int start;

    cout << "Enter number of vertices: ";
    cin >> n;

    cout << "Enter weighted adjacency matrix:\n";
    cout << "(Enter 0 if there is no edge)\n";

    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= n; j++)
            cin >> graph[i][j];
    }

    cout << "\nEnter source vertex: ";
    cin >> start;

    dijkstra(start);

    return 0;
}
```